

Лабораторная работы

Основы языка PL/SQL

Цель работы: Получить общее представление о PL/SQL и познакомиться с основными понятиями языка. Изучить реляционные свойства PL/SQL, включая синтаксис языка, типы данных, способы использования SQL, инструкции DML, а также особенности работы с транзакциями. Рассмотреть синтаксис создания подпрограмм (процедур и функций), которые могут храниться и выполняться на стороне сервера БД.

Теоретический материал:

PL/SQL - это развитый язык программирования, используемый для доступа к базам данных Oracle из различных сред. PL/SQL интегрирован с сервером базы данных, поэтому программы PL/SQL обрабатываются быстро и эффективно. Этот язык доступен и в некоторых клиентских инструментальных средствах Oracle.

Язык программирования, используемый для доступа к реляционным базам данных, называется языком структурированных запросов (SQL, Structured Query Language). SQL — гибкий и эффективный язык, все средства которого служат для манипулирования реляционными данными и для их исследования. Например, с помощью SQL-оператора можно удалить из базы данных сведения обо всех студентах (students), фамилия (*last_name*) которых 'Иванов':

```
delete from students  
where last_name = 'Иванов'
```

PL/SQL означает Procedural Language/SQL (процедурный язык/SQL). Как видно из названия, PL/SQL расширяет возможности SQL, добавляя в него такие конструкции процедурных языков, как:

- переменные и типы данных (как предварительно определенные, так и определяемые пользователями);
- управляющие структуры, такие как условные операторы и циклы;
- процедуры и функции;
- объектные типы и методы (OracleS и выше);

Процедурные конструкции объединяются с Oracle SQL, что дает в результате структурированный и эффективный язык программирования. Предположим, что требуется изменить профилирующую дисциплину некоего студента. Если записи о нем нет, нужно создать новую запись. Это можно сделать при помощи следующего кода PL/SQL:

```
Declare
/*Объявление переменных */
  v_New VARCHAR2(10):= 'History';
  v_FirstName VARCHAR2(10):= 'Scott';
  v_LastName VARCHAR2(10):= 'Urman';
Begin
/* Обновление таблицы students */
  UPDATE students
  SET major= v_New,
      WHERE first_name= v_FirstName and last_name=
v_LastName;
/* Проверка, найдена ли запись. Если нет, нужно ее ввести */
  If SQL%NOTFOUND THEN
    INSERT INTO Students (ID, major, first_name, last_name)
    VALUES (student_sequence.NEXVAL, v_New, v_first_name,
            v_last_name);
  End If;
End.
```

В этом примере содержатся два разных SQL-оператора (UPDATE (обновить) и INSERT (ввести)), а также SQL-конструкции (операторы объявления переменных и условный оператор IF).

Блочная структура

Базовой единицей PL/SQL является блок (block). Все программы PL/SQL строятся из блоков, которые могут быть вложены один в другой. Как правило, в программе каждый блок выполняет определенную логическую единицу работы, что помогает отделить друг от друга различные задачи. Блок имеет следующую структуру:

```
[ DECLARE
  -- описание переменных, констант и пользовательских типов
  данных ]
```

BEGIN

*-- тело основной программы, в которую могут вкладываться
другие блоки (в них также можно вкладывать блоки и т.д.)*

[EXCEPTION

*-- драйверы для обработки исключительных ситуаций в
программе]*

END;

Программы PL/SQL могут быть неименованными (анонимными блоками), но чаще всего используются именованные программы: процедуры, функции, пакеты и триггера. Обязательным в представленной схеме является только выполняемый раздел. Он должен содержать по меньшей мере один выполняемый оператор. Разные разделы блока в программе PL/SQL предназначены для выполнения различных функций.

В разделе объявлений размещаются объявления всех переменных, курсоров и типов, используемых данным блоком. В этом разделе могут быть объявлены также локальные процедуры и функции. Такие подпрограммы будут доступны только в пределах блока. В выполняемом разделе осуществляется работа блока. В этом разделе могут находиться как процедурные, так и SQL-операторы.

Раздел блока по обработке исключительных ситуаций используется для реагирования на ошибки, которые встречаются во время выполнения программы. При отделении программного кода, предназначенного для обработки ошибок, от основного тела программы структура программы становится гораздо понятнее.

Если раздел объявлений отсутствует, выполнение блока начинается с ключевого слова BEGIN. Если отсутствует раздел исключительных ситуаций, ключевое слово EXCEPTION пропускается и блок заканчивается ключевым словом END и точкой с запятой.

Язык программирования PL/SQL разработан на основе языка третьего поколения Ada. Многие конструкции, применяемые в Ada, можно найти в PL/SQL. Одним из общих свойств этих языков является их блочная структура. Из Ada заимствованы также обработка исключительных ситуаций, синтаксис объявления процедур и функций, модули.

Типы данных, доступные в PL/SQL

Кроме типов данных Oracle7, PL/SQL поддерживает несколько дополнительных типов данных и позволяет использовать в своих конструкциях некоторые основные типы с большим диапазоном.

Тип данных	Описание
BINARY_INTEGER	Этот тип данных и его подтипы NATURAL и POSITIVE применяются для создания переменных и констант, которые хранят число со знаком. Двоичные целые числа могут принимать значения в диапазоне от -2 в 31 степени до 2 в 31 степени минус 1.
BOOLEAN	Принимается для создания переменных и констант, в которых хранятся логические значения TRUE и FALSE.
CHAR	Есть подтипы CHARACTER и STRING. Максимальный размер 32767.
NUMBER	Есть подтипы DEC, DECIMAL, DOUBLE, PRECISION, FLOAT, INT, INTEGER, NUMERIC, REAL и SMALLINT.
RECORD	Используется для создания пользовательских типов записей базы данных.
TABLE	Служит для создания табличных типов данных PL/SQL.
VARCHAR2	Есть подтип VARCHAR. Максимальный размер 32767.
tab%ROWTYPE	Используется для определения типа данных записи по типу данных столбцов таблицы, к имени которой (tab) приписан суффикс %ROWTYPE.
col%TYPE	Используется для определения типа данных столбца или переменной по типу данных другого столбца или переменной, к имени которого или которой (col) приписан суффикс %TYPE.

Переменные, константы и выражения

В программах PL/SQL могут использоваться переменные и константы, описываемые в разделе DECLARE с помощью конструкции вида:

```
variable_name [CONSTANT] type_name [NOT NULL] [  
{ := | DEFAULT } expr ]
```

Например:

```
birthdate      DATE;  
emp_count      SMALLINT := 0;  
emp_count      SMALLINT DEFAULT 0;  
acct_id        VARCHAR2(5) NOT NULL := 'AP001';  
pi             CONSTANT REAL := 3.14159;  
area           REAL := pi * radius**2;  
valid_id       BOOLEAN;  
valid_id       VARCHAR2(5); /* недопустимое вторичное описание  
                           valid_id*/  
  
i, j, k        SMALLINT; /* нельзя описывать список; надо:  
                           i SMALLINT; j SMALLINT;  
                           k SMALLINT;*/  
  
credit         REAL(7,2);  
debit          credit%TYPE; /* тип данных аналогичный типу данных  
                           "credit" */
```

Анонимный блок

Анонимные блоки - это блоки, которые не имеют имени. Анонимные блоки не могут быть вызваны другими блоками, так как у них нет имени, на которое можно ссылаться.

```
[локальные объявления]  
BEGIN  
    исполняемые предложения  
[EXCEPTION  
    обработчики исключений  
END [имя];
```

Процедуры

Процедура - это подпрограмма, которая выполняет специфическое действие. Вы пишете процедуры, используя следующий синтаксис:

```
PROCEDURE имя [ (параметр [, параметр, ...]) ] IS
    [локальные объявления]
BEGIN
    исполняемые предложения
[EXCEPTION
    обработчики исключений]
END [имя];
```

где каждый "параметр" имеет следующий синтаксис:

```
имя_перем [IN | OUT | IN OUT] тип_данных [{:= | DEFAULT} знач]
```

В отличие от спецификатора типа данных в объявлении переменной, спецификатор типа данных для параметра не может иметь ограничений. Например, следующее объявление name незаконно:

```
PROCEDURE ... (name CHAR(20)) IS -- незаконно; должно
быть CHAR
BEGIN ... END;
```

Процедура имеет две части: спецификацию и тело. Спецификация процедуры начинается с ключевого слова PROCEDURE и заканчивается именем процедуры или списком параметров. Объявления параметров необязательны. Если процедура не принимает параметров, скобки также не кодируются.

Тело процедуры начинается с ключевого слова IS и заканчивается ключевым словом END, за которым может следовать имя процедуры. Тело процедуры состоит из трех частей: декларативной части, исполняемой части и необязательной части обработки исключений.

Декларативная часть содержит локальные объявления, которые помещаются между ключевыми словами IS и BEGIN. Ключевое слово DECLARE, которое начинает декларативную часть в анонимном блоке PL/SQL, здесь не используется. Исполняемая часть содержит предложения, которые помещаются между ключевыми словами BEGIN и EXCEPTION (или

END). В исполняемой части процедуры должно быть хотя бы одно предложение. Часть обработки исключений содержит обработчики исключений, которые помещаются между ключевыми словами EXCEPTION и END.

Рассмотрим процедуру raise_salary, которая увеличивает жалование сотрудника:

```
PROCEDURE raise_salary (emp_id NUMBER, increase REAL) IS
    current_salary REAL;
    salary_missing EXCEPTION;
BEGIN
    SELECT sal INTO current_salary FROM emp
        WHERE empno = emp_id;
    IF current_salary IS NULL THEN
        RAISE salary_missing;
    ELSE
        UPDATE emp SET sal = sal + increase
            WHERE empno = emp_id;
    END IF;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        INSERT INTO emp_audit VALUES (emp_id, 'No such number');
    WHEN salary_missing THEN
        INSERT INTO emp_audit VALUES (emp_id, 'Salary is null');
END raise_salary;
```

При своем вызове эта процедура принимает номер сотрудника и сумму, на которую увеличивается его жалование. Она использует номер, чтобы выбрать текущее жалование из таблицы базы данных emp. Если номер сотрудника не найден или текущая зарплата не определена, возбуждается исключение. В противном случае жалование обновляется.

Процедура вызывается как предложение PL/SQL. Например, процедура raise_salary может быть вызвана так:

```
raise_salary(emp_num, amount);
```

Функции

Функция - это подпрограмма, которая вычисляет значение. Функции структурируются так же, как и процедуры, с той разницей, что функции содержат фразу RETURN. Вы пишете функции, используя синтаксис

```
FUNCTION имя [ (аргумент [, аргумент, ...]) ] RETURN тип_данных IS
    [локальные объявления]
BEGIN
    исполняемые предложения
[EXCEPTION
    обработчики исключений]
END [имя];
```

где каждый "аргумент" имеет следующий синтаксис:

```
имя_переменной [IN | OUT | IN OUT] тип_данных [{:= | DEFAULT} знач]
```

В отличие от спецификатора типа данных в объявлении переменной, спецификатор типа данных для параметра не может иметь ограничений.

Как и процедура, функция имеет две части: спецификацию и тело. Спецификация функции начинается с ключевого слова FUNCTION и заканчивается фразой RETURN, которая специфицирует тип результирующего значения. Объявления аргументов необязательны. Если функция не принимает аргументов, скобки также не кодируются.

Тело функции начинается с ключевого слова IS и заканчивается ключевым словом END, за которым может следовать имя функции. Тело функции состоит из трех частей: декларативной части, исполняемой части и необязательной части обработки исключений. Декларативная часть содержит локальные объявления, которые помещаются между ключевыми словами IS и BEGIN. Ключевое слово DECLARE, которое начинает декларативную часть в анонимном блоке PL/SQL, здесь не используется. Исполняемая часть содержит предложения, которые помещаются между ключевыми словами BEGIN и EXCEPTION (или END). В исполняемой части процедуры должно встретиться хотя бы одно предложение RETURN. Часть обработки исключений содержит обработчики исключений, которые помещаются между ключевыми словами EXCEPTION и END.

Рассмотрим функцию `sal_ok`, которая определяет, не выходит ли жалование для данной должности за допустимые границы:

```
FUNCTION sal_ok (salary REAL, title REAL) RETURN BOOLEAN IS  
  min_sal REAL;  
  max_sal REAL;  
BEGIN  
  SELECT losal, hisal INTO min_sal, max_sal FROM sals  
    WHERE job = title;  
RETURN (salary >= min_sal) AND (salary <= max_sal);  
END sal_ok;
```

При своем вызове эта функция принимает жалование сотрудника и его должность. Она использует должность, чтобы выбрать границы оклада из таблицы базы данных `sals`. Если жалование выходит за эти границы, `sal_ok` возвращает `FALSE`; в противном случае `sal_ok` возвращает `TRUE`.

Функция вызывается как часть выражения. Например, функция `sal_ok` может быть вызвана так:

```
IF sal_ok(new_sal, new_title) THEN  
  ...  
END IF;  
  
  ...  
promotable := sal_ok(new_sal, new_title) AND (rating > 3);
```

В обоих случаях идентификатор функции является выражением, которое заменяется своим значением.

Лабораторная работа 1 (простые процедуры)

1. Создайте таблицу MyTable(id number, val number).
2. Напишите анонимный блок, который записывает в таблицу MyTable 10 000 целых случайных записей.
3. Напишите собственную функцию, которая выводит TRUE если четных значений val в таблице MyTable больше, FALSE если больше нечетных значений и EQUAL, если количество четных и нечетных равно.
4. Напишите функцию, которая по введенному значению ID, сгенерирует и выведет в консоль текстовое значение команды insert для вставки указанной строки.
5. Написать процедуры, реализующие DML операции (INSERT, UPDATE, DELETE) для указанной таблицы.
6. Создайте функцию, вычисляющую общее вознаграждение за год. На вход функции подаются значение месячной зарплаты и процент годовых премиальных. В общем случае общее вознаграждение= (1+ процент годовых премиальных)*12* значение месячной зарплаты. При этом предусмотреть что процент вводится как целое число, и требуется преобразовать его к дробному. Предусмотреть защиту от ввода некорректных данных.

Лабораторная работа 2 (триггеры)

1. Построить две таблицы STUDENTS и GROUPS реализующих соответственно справочник студентов и справочник групп:

Поле	Тип	комментарий
STUDENTS		
ID	Number	Код студента
NAME	VARCHAR2	Имя студента
GROUP_ID	Number	Код группы
GROUPS		
ID	Number	Код группы
NAME	VARCHAR2	Название группы
C_VAL	Number	Количество студентов в группе

2. Реализовать триггеры для таблиц задания 1: проверку целостности (проверка на уникальность полей ID), генерацию автоинкрементного ключа и проверку уникальности для поля GROUP.NAME.

3. Реализовать триггер, реализующий Foreign Key с каскадным удалением между таблицами STUDENTS и GROUPS.

4. Реализовать триггер, реализующий журналирование всех действий над данными таблицы STUDENTS

5. Исходя из данных предыдущей задачи, реализовать процедуру для восстановления информации на указанный временной момент и на временное смещение.

6. Реализовать триггер, который в случае изменения данных в таблице STUDENTS будет соответственно обновлять информацию C_VAL таблицы GROUPS.

Лабораторная работа 3 (работа со схемами данных)

1. Написать процедуру/функцию, на вход которой подаются два текстовых параметра (`dev_schema_name`, `prod_schema_name`), которые являются названиями схем баз данных (условно схема для разработки(Dev) и промышленная схема(Prod)). На выход процедура должна предоставить перечень таблиц, которые есть в схеме Dev, но нет в Prod, либо в которых различается структура таблиц. Наименования таблиц должны быть отсортированы в соответствии с очередностью их возможного создания в схеме prod (необходимо учитывать foreign key в схеме). В случае закольцованных связей выводить соответствующее сообщение.
2. Доработать предыдущий скрипт с учетом возможности сравнения не только таблиц, но и процедур, функций, индексов, пакетов.
3. Доработать предыдущий скрипт с генерацией ddl-скрипта на обновление объектов, а также с учетом необходимости удаления в схеме prod объектов, отсутствующих в схеме dev.

Лабораторная работа №4 (redis)

Дана схема реляционной базы данных

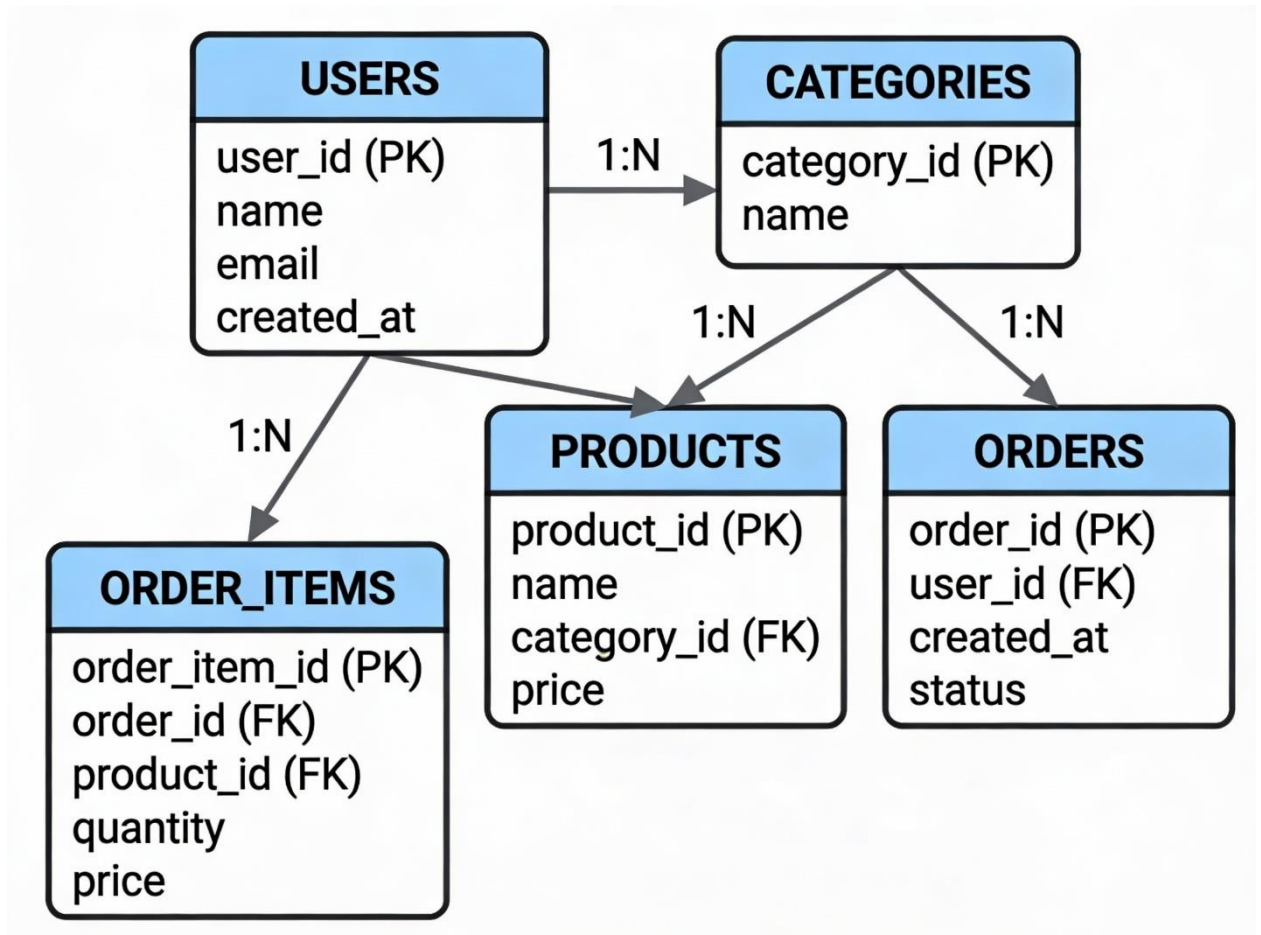


Диаграмма показывает:

- **Users** (пользователи) с `user_id` как первичный ключ
- **Categories** (категории товаров) с `category_id` как первичный ключ
- **Products** (товары) с внешним ключом `category_id` на **Categories**
- **Orders** (заказы) с внешним ключом `user_id` на **Users**
- **OrderItems** (позиции заказов) с внешним ключом `order_id` на **Orders** и `product_id` на **Products**, а также собственным первичным ключом `order_item_id` для упрощения миграции

Все связи реализованы как один-ко-многим (один пользователь → много заказов, один заказ → много позиций и т.д.).

Прилагаются файлы с данными для импорта.

Необходимо:

- перечислить основные **use cases**:
например, «вывести последние заказы пользователя», «посчитать топ-N

популярных товаров», «найти все товары категории», «посчитать выручку по пользователю» и т.д.;

- отметить операции, где в SQL использовались **JOIN, GROUP BY, ORDER BY, LIMIT**.
- Спроектировать ключи и структуру базы данных Redis
 - какие типы данных использовать:
 - Hash для сущности (user:{id}, product:{id});
 - Set для множества (user:{id}:orders, category:{id}:products);
 - Sorted Set для топов (products:by_sales, products:by_rating);
 - List для «ленты» (последние заказы, события).
 - структуру ключей:
 - соглашения вида entity:{id}, entity:{id}:subentity, index:{field}:{value} и т.п.;
- Оформить **таблицу соответствий** «таблица/запрос в SQL → структура данных и ключи в Redis».
- На основе предоставленных данных в CSV написать **скрипт миграции в Redis**:
 - прочитать данные из файлов/БД;
 - создать Hash-объекты для сущностей (пользователи, товары, заказы);
 - заполнить необходимые множества/списки/сортированные множества для индексов и агрегатов.
- Проверить через redis-cli наличие и корректность ключей, (например KEYS user:*, HGETALL user:123, ZRANGE products:by_sales 0 9 WITHSCORES)
- Для указанной предметной области задать **набор бизнес-операций**, которые нужно реализовать с использованием Redis.
 - Получить последние 10 заказов пользователя
 - хранить идентификаторы заказов в List/Sorted Set user:{id}:orders;
 - реализовать функцию (на выбранном языке) или последовательность команд в redis-cli, возвращающую заказы с деталями.
 - «Топ N товаров по количеству продаж / по выручке»
 - использовать Sorted Set products:by_sales (score = количество проданных единиц) и/или products:by_revenue (score = суммарная выручка);
 - реализовать функции get_top_products_by_sales(n) и get_top_products_by_revenue(n).
 - «Товары по категории + фильтрация по цене»
 - Set category:{id}:products → множество идентификаторов;
 - отдельный Sorted Set products:by_price или хранение цены в Hash + фильтрация приложением.
 - «Похожие пользователи / рекомендации»

- хранить множество просмотренных/купленных товаров пользователем (user:{id}:viewed, user:{id}:purchased как Sets);
 - использовать операции множеств (SINTER, SUNION, SDIFF) для нахождения пересечений.
- Выбрать одну или две «тяжёлые» операции (например, агрегаты по продажам) и реализовать:
 - ключ кэша, например cache:top_products:by_sales:{N};
 - заполнение кэша (вычисление значения и SET key value EX 60);
 - последующее чтение с проверкой наличия (GET → при null пересчитать и продлить кэш)

Лабораторная работа № 5 (Redis, очереди)

Лабораторная работа предназначена для определения возможных ролей Redis в архитектуре: таких как очереди, rate limiting, real-time, persistence, масштабирование (на базовом уровне).

1. Необходимо реализовать очередь задач с использованием Lists

1.1 Спроектировать очередь фоновых задач:

- ключ очереди, например queue:tasks;
- формат задачи (JSON-строка как значение, ID как часть payload).

1.2 Реализовать:

- producer: добавляет задания в очередь с помощью LPUSH/RPUSH;
- consumer (воркер): читает задания BRPOP/BLPOP, обрабатывает, логирует результат.

1.3 Обработать как минимум два типа задач (например, отправка email / обработка заказа / запись лога).

2 реализовать **ограничение количества запросов к API** для одного пользователя/токена, например: не более 100 запросов в 10 минут.

2.1 Простой счетчик с TTL:

- ключ rate:{user_id}:{window_start_timestamp};
- при каждом запросе:
 - INCR key;
 - если INCR вернул 1 — поставить EXPIRE key window_size;
 - если значение > limit → запрос отклонить.

2.2 Скользящее окно

- хранить timestamps в List или Sorted Set и периодически очищать старые;
- реализовать логику через Lua-скрипт для атомарности.

Лабораторная работа №6 (MongoDB основы)

Исходя из схемы БД и данных, представленных в Лабораторной работе №4

1. Написать скрипт (на любом языке), который читает CSV файлы, создает коллекции `users`, `categories`, `products`, `orders`
2. Написать запросы, проверяющие целостность данных:
 - a. Подсчитать количество документов в каждой коллекции
 - b. Убедиться, что все заказы имеют валидные `user_id` (проверка через `$lookup`)
 - c. Найти заказы с общей суммой > 1000 (проверка расчетов)
3. Реализовать функции, аналогичные SQL:
 - a. `get_user_orders(user_id)` - аналог `SELECT * FROM orders WHERE user_id = ?`
 - b. `get_order_with_details(order_id)` - аналог `JOIN orders + order_items + products`
 - c. `get_top_products_by_revenue(limit)` - аналог `GROUP BY product_id ORDER BY SUM(quantity*price) DESC`
4. Провести проверку производительности
 - a. Создать индексы: `db.orders.createIndex({ user_id: 1, created_at: -1 })`
 - b. Сравнить время выполнения запроса до и после создания индекса (использовать `explain("executionStats")`)

Лабораторная работа №7 (MongoDB фреймворк агрегации)

Реализовать:

1. набор агрегационных конвейеров для бизнес-метрик:
 - a. Ежемесячная выручка по категориям (аналог SQL: GROUP BY category, month)
 - b. Корзиночный анализ (market basket analysis) - найти товары, которые часто покупают вместе
 - c. RFM-анализ клиентов (Recency, Frequency, Monetary)
2. Провести анализ производительности агрегации
 - a. Создать materialized view с \$merge или \$out для часто используемых агрегатов
 - b. Сравнить время выполнения "живой" агрегации vs чтение из pre-aggregated коллекции
3. Реализация сложного бизнес-процесса, например: "Перемещение товара между складами с обновлением инвентаря и созданием записи в истории"
4. Провести анализ уровней изоляции
 - a. Смоделировать гонку между двумя конкурентными транзакциями (например, два пользователя пытаются купить последний товар)
 - b. Показать, как writeConflict предотвращает некорректное состояние